# A Forwards-Only, Asynchronous Analog Architecture for Predictive-Coding Networks: Local Transpose and Sigma–Delta Weight Update for On-Die Supervised Learning

Saul Dobney*

2026

**Abstract**

Analog architectures for predictive-coding networks benefit from in-memory computation which removes the weight-movement cost that dominates digital neural inference, but currently there are no established mechanisms for *supervised training in place* that avoid a backward pass, per-device calibration, or a host in the loop. We present an analog predictive-coding architecture that trains in place under three invariants — no per-device calibration, single direction signal paths, and no global clock — and show they cost essentially no accuracy. This is accomplished by a separated weights and error design, computing each weight matrix's transpose on the chip that owns it and a sigma–delta weight update via a leaky-jug error capacitor and a threshold comparator, whose cumulative quantisation error is bounded independently of the number of updates (the first-order sigma–delta property). In this design a per-update step far below one weight LSB (0.019 LSB) drives a coarse analog cell with no per-synapse digital accumulator, which means the comparator may be wrong at a substantial rate, but device mismatch appears only as a benign per-synapse spread of the learning rate. On EMNIST letters with a 48-chip topology the forwards-only rule reaches 82.50% against a full-backpropagation ceiling of 82.85% on identical topology, and a bit-faithful hardware model stays within the run-to-run noise band of its floating-point ceiling. All results are pre-silicon.

## 1 Introduction

The energy cost of a large neural network is dominated by moving weights, not by multiplying them. On conventional hardware the weights reside in memory and are streamed through a fixed arithmetic fabric for every input; the resulting data movement, rather than the multiply–accumulate itself, sets the power budget [2, 3, 4]. Analog in-memory computation addresses this directly. If a weight is stored as charge on a capacitor and the multiplication is performed in place as a transconductance, with summation by Kirchhoff's current law, then the weights never move and the dominant cost is removed by construction. Part I of this study [1] demonstrated such a substrate for predictive-coding networks: a compact analog cell that stores a weight as a capacitor voltage and learns

*Correspondence: saul.dobney@dobney.com. Part I of this study [1] presents the analog cell and its unsupervised learning; this is Part II. Design files, RTL, SPICE netlists and simulation scripts are available at https://github.com/dobneyresearch/PCNchip_with_leakyjug_learning.

*unsupervised* through a Hebbian rule [5], reaching 83% on MNIST and 64% on EMNIST letters with a linear read-out over the learned features.

This paper addresses whether the substrate can be driven to task accuracy on real data by *supervised* learning, and what the learning hardware would have to look like. We develop both the mathematics and the hardware design in parallel as hardware adds real function constraints that affect factors like signal size and timing. Supervised training, as normally formulated, propagates a signed error backwards through the transpose of every forward weight matrix, in step with the forward pass. In digital hardware this is routine. In an analog in-memory substrate, an analog transconductor is a one-way device, and current cannot simply be driven backwards through it. The alternatives are a second, reverse analog datapath — doubling the analog design problem and requiring the two directions to be held in alignment — or a digital copy of every weight held wherever the transpose is computed, which restores the weight traffic that motivated the substrate. Either way, the resulting system needs a global schedule to keep forward and backward passes coherent, which is the property that makes large analog neuromorphic fabrics fragile and hard to grow.

This barrier is not unique to predictive coding; it is the central difficulty of the whole analog-learning field, approached from several directions. Analog in-memory training on non-volatile memory has reached digital-equivalent accuracy [4], but it runs conventional backpropagation — transposed backward pass included — and depends on device-level symmetry of the weight update. Mixed-signal spiking neural networks (SNNs) take the opposite premise: following the biological observation that neurons *integrate inputs as an analog sum but communicate with spikes* [6], they compute in analog and route events on an asynchronous digital fabric [7, 8, 9]. Their unsolved problem is training: device mismatch and low precision distort weights and activations, so competitive accuracy is obtained either by per-neuron calibration or by training the substrate *in the loop* with a host — for example surrogate-gradient descent on BrainScaleS-2, where "learning self-corrects for device mismatch" [6]. On-chip local rules that avoid the host [10, 11] remain Hebbian or spike-timing based and do not reach backpropagation-grade credit assignment on deep tasks. The common thread is that making a mismatched analog substrate learn well has, so far, required either calibration, a backward pass, or a host in the training loop.

Within predictive coding proper, Whittington and Bogacz [12] establish the relationship to backpropagation that motivates the approach, and Millidge et al. [13] survey subsequent variants; recent work has also reported synthesisable RTL realisations of PC networks [14]. What has been missing is an analog PC design that trains in place *without* reintroducing the backward pass, the host, or the calibration step. This paper reports such an architecture, together with its validation. We adopt three constraints as design invariants rather than objectives, and do not relax them:

1. **Forwards-only.** Signal paths are one-way. No reverse analog channel exists, and no two-directional pipeline has to be kept in alignment.

2. **Robust.** The system must tolerate device mismatch, noise, and lost messages, and must not depend on per-device calibration or on a control loop holding an operating point.

3. **Asynchronous.** No global clock is distributed across the network. Chips act on messages as they arrive; timing is local.

The contributions follow the $\boldsymbol{W}\&\boldsymbol{E}$ organisation — separate weight and error stores linked along a one-way signal path:

- An architecture in which the chip is an autonomous predictive-coding *level*: it holds one copy of its weights $\boldsymbol{W}$ and reads that copy three ways — forward prediction, transpose relay, and

error accumulation — so that no weight ever leaves the die and every weight update is a local transaction (Sec. 3).

- *Transpose-at-source*: computing $\boldsymbol{W}^\top \boldsymbol{\delta}$ on the chip that owns $\boldsymbol{W}$, which eliminates the remote weight copy together with its coherence traffic and its synchronisation barrier, and reduces the router to summation and division (Secs. 3, 5).
- A sigma–delta weight update in the error store $\boldsymbol{E}$ (the *leaky jug*), realised as a leaking capacitor and a shared comparator. The modulator is a standard structure; the contribution is its use as the weight-update path and the training-specific consequences of its bounded quantisation error — why a learning rate of 0.019 weight LSB can drive an 8-bit analog cell with no per-synapse digital accumulator, why the comparator's decisions may be wrong at a substantial rate without corrupting learning, and why device mismatch acts only as a benign per-synapse learning-rate spread (Secs. 4, 6).
- Validation on EMNIST letters showing the forwards-only rule matches backpropagation on the same topology, survives a bit-faithful hardware model and hostile device mismatch, with reported negative results (Secs. 6, 7), and an explicit positioning against NVM-based and spiking analog designs (Sec. 8).

# 2 Predictive coding and the analog-neuromorphic context

## 2.1 The $\boldsymbol{W}$ & $\boldsymbol{E}$ principle

Predictive coding models perception as inference in a hierarchical generative model: each level emits a prediction of the level below and retains the discrepancy as an error, and both representations and weights are adjusted to reduce it [15, 16]. Its defining structural commitment, and the one we build on, is that representation and error are carried by *separate* populations interacting only locally, one level apart. We refer to this as the $\boldsymbol{W}$&$\boldsymbol{E}$ organisation: activations are propagated forward through the weights $\boldsymbol{W}$, errors are accumulated in a separate store $\boldsymbol{E}$, and when $\boldsymbol{E}$ crosses a threshold the corresponding weight in $\boldsymbol{W}$ is adjusted. Two properties make this a serious hardware substrate rather than a biological analogy. It is *locally computable* — every quantity a synapse needs is present at that synapse, so an update can be performed in place rather than by a global gather — and it has a *defensible relation to backpropagation*: a PC network with local Hebbian plasticity computes updates that converge to those of backpropagation when the output error is small relative to other activity [12]. Although we started by testing the W+E architecture with contrastive Hebbian learning, the scheme we describe learns by approximating the backpropagation gradient due to challenges making CHL work as a hardware solution.

## 2.2 Four obstacles to an analog realisation of CHL

Canonical predictive coding using CHL, and in the SNN tradition it is adjacent to, present four obstacles to an analog implementation.

**C1: the framework is bidirectional.** A Predictive Coding level sends predictions *down* and receives errors *up*. Realised literally in analog silicon, this is two signal paths through the same array in opposite directions — a physically bidirectional cell or a duplicated datapath. Both are expensive, and both reintroduce the alignment problem between the two directions; a unit

computing the error arriving from above must, moreover, have access to the weights of the level above, which if it does not own them must be copied and kept coherent.

**C2: inference is iterative and clock-based.** Standard Predictive Coding has a free and a clamped phase that relaxes the representation to equilibrium before updating the weights, requiring a settling phase of unspecified length. A settling phase implies a schedule, and a schedule across many chips implies a clock. Combined with signal bidirectionality this adds a synchronisation constraint which, in our experiments, became fragile and parameter dependent — in direct tension with asynchronicity and robustness.

**C3: the update quantum is tiny.** Analog storage is sensitive to delta size when updating. An effective learning rate for this task is $\eta \approx 3 \times 10^{-4}$, while the analog weight cell resolves 1/64 per code. A single update is therefore $\approx 0.019$ of one weight LSB: rounded to the cell's resolution it is exactly zero, roughly fifty times in succession. Writing a correct gradient to a coarse analog cell naively does not slow learning down; it stops it altogether. Deltas can be passed through a high-precision digital *master weight* per synapse that accumulates sub-LSB updates and is periodically written down but our hardware experiments suggested this requires $\sim 18$ bits of accumulator plus a velocity register per synapse, a per-synapse digital memory larger than the analog weight it exists to serve.

**C4: spiking is treated as the system, not a communication protocol.** Conventional SNNs adopt spiking as a defining constraint of the whole design, so that computation, state and communication are all expressed in spikes. We separate computation from communication: the analog sum is the computation, and a spike is one possible *encoding* of a message on the interconnect, not the computational primitive. This makes the choice of signalling — graded packet, event, or spike — a property of the fabric rather than of the algorithm, and it is what allows the same design to be clock-free without committing to spike-timing codes.

The design resolves C1 by relocating the transpose onto the chip that owns the weights $\boldsymbol{W}$ (Sec. 3.4), C2 by abandoning relaxation in favour of a single forward evaluation per sample (Sec. 4), C3 by replacing the digital master weight with a sigma–delta accumulator $\boldsymbol{E}$ (Sec. 4.3), and C4 by making the message class a configurable property of the interconnect (Sec. 5).

# 3 Architecture

## 3.1 Design principles

The final architecture is a mixed analog-digital design that allows for flexible topologies. Core analog compute is done via capacitance, but weight values are held in SRAM to protect weights from decay on the capacitor.

The design rests on a small set of commitments a PCN or SNN reader can hold throughout. (i) Representation and error are separate populations ($\boldsymbol{W}$&$\boldsymbol{E}$). (ii) Each chip keeps exactly one copy of its weights in SRAM and reads it three ways — forward prediction, transpose relay, and error accumulation. (iii) The inter-chip router only *gathers* (sum and divide); it holds no weights, so the network topology is a soft, reconfigurable property of the router rather than fixed wiring. (iv) All signalling is forward-directed; there is no reverse analog channel and no counter-propagating pipeline. (v) The update approximates the backpropagation gradient (Sec. 2), not contrastive Hebbian learning, and is driven by a minimal global scalar — a "happiness" signal derived from

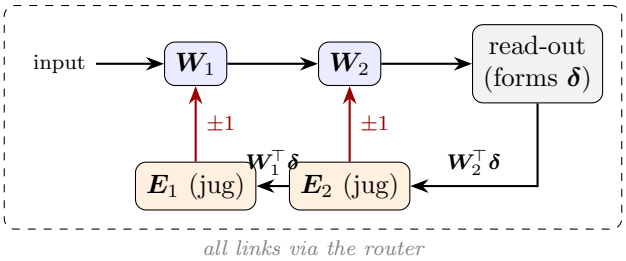


**Figure 1: The learning loop in one picture.** Forward activations pass input $\rightarrow \boldsymbol{W}_1 \rightarrow \boldsymbol{W}_2 \rightarrow$ read-out, which forms the output error $\boldsymbol{\delta}$. The error is relayed *backward through the network but forward in signalling*: each layer computes $\boldsymbol{W}^\top \boldsymbol{\delta}$ on the chip that owns $\boldsymbol{W}$ and injects it into that layer's error store $\boldsymbol{E}$ (a leaky jug), which writes single-code updates into $\boldsymbol{W}$. All links are mediated by the router. No analog signal flows in reverse.

the read-out's margin — that gates learning severity without carrying per-unit information. (vi) Every element is chosen to be synthesisable or fabricable in an open 130 nm process (Sky130A). We describe the design from the unit that matters most — the chip and its single weight memory — outward to the network. Equations are deferred to Sec. 4.

### 3.2 The substrate and its granularity in hardware

The compute element, inherited from Part I [1], is a five-transistor operational transconductance amplifier: an 8-bit code held in SRAM (one step per 1/64 of the weight range) is converted to a voltage on a 200 fF capacitor, which sets the tail current of a differential pair and hence the transconductance by which the cell multiplies its input. Column currents sum on a shared node by Kirchhoff's current law, and an 8-bit SAR ADC digitises the result. The array unit is a $16 \times 16$ *cell* (256 such multipliers); a *chip* carries four cells; a *router* mediates all traffic between chips. Routing is performed at cell granularity and never finer, which keeps the routing state $O(\text{cells}^2)$ rather than $O(\text{synapses}^2)$; fine-grained mixing is the job of the destination cell's own weight matrix, not of the network. Because the router, not the wiring, defines which cell feeds which, the topology is reconfigurable and open to experiment.

**The weight cell is non-linear, and is linearised by pre-distortion.** The five-transistor cell's effective weight — the differential-pair transconductance — is not linear in the stored code: it peaks near mid-range and falls above it, so the upper half of the nominal range is compressed and, at the extreme, inverted. This is a property of the cell, measured across the full code range in SPICE (a single-operating-point check cannot see it). The design linearises it in two steps: the tail device is sized to restore monotonicity, and the weight DAC is driven through a fixed pre-distortion table (Fig. 2) so that stored code maps monotonically to effective weight. With the measured cell curve and this pre-distortion in place, accuracy is 81.46% — within the run-to-run noise band of the 81.96% obtained under an ideal linear cell (Sec. 6) — so the bit-faithful model's linear-weight treatment is a consequence of the pre-distortion, not an assumption.

A layer is thus *factored* across chips: in the topology evaluated here each first-layer chip maps 48 inputs to 16 outputs as three sequential $16 \times 16$ pages, and each deeper chip gathers a fixed small number of upstream chips. This factoring is an architectural commitment with a measurable accuracy cost, which we quantify in Sec. 6 rather than leave implicit.

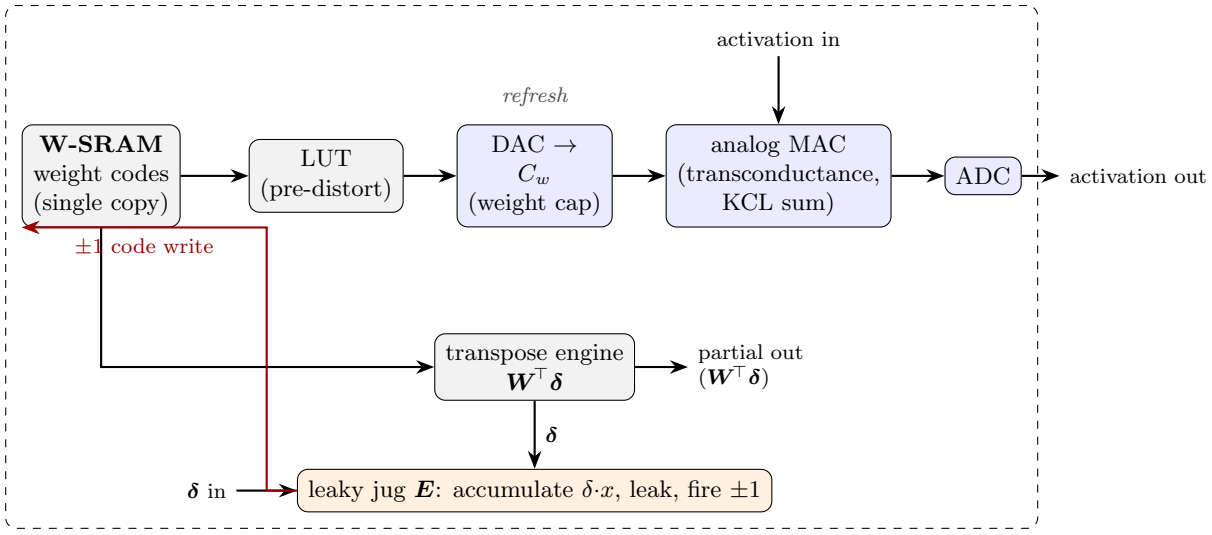


**Figure 2: The chip: one weight memory with three consumers.** The W-SRAM (grey) is the single copy of the weights. It drives the analog forward path (blue) through a pre-distortion LUT and a weight DAC; it is read a second time by the on-die transpose engine, which relays error to the preceding layer; and it is written by the leaky jug (orange), which converts accumulated error into single ±1 code steps. The incoming error $\boldsymbol{\delta}$ is an ordinary forward-directed message and is consumed twice, by the transpose and by the accumulator. No analog signal flows in reverse.

### 3.3 One weight memory, three consumers

The central structural decision is that each chip holds exactly one copy of its weights, as digital codes in a local SRAM, and that three functionally distinct consumers read that same copy (Fig. 2).

1. **Forward prediction ($\boldsymbol{W}$).** The code is mapped through a pre-distortion look-up table to the weight DAC and the analog array multiplies the incoming activation. This is the in-memory forward pass.

2. **Backward relay ($\boldsymbol{W}^\top$).** The same codes are read by a small digital engine that computes $\boldsymbol{W}^\top \boldsymbol{\delta}$ for the incoming error and emits the result as a partial sum. This digital engine could be substituted by a cross-array read as analog (see below).

3. **Error accumulation ($\boldsymbol{E}$).** The incoming error, scaled by the locally latched forward activity, is injected onto a per-synapse capacitor; a threshold crossing writes a single ±1 step back to the SRAM.

Three consequences follow, and they are the reasons for the decision. There is no second copy of any weight, hence no coherence protocol and no weight traffic. Every quantity a weight update requires is present on the die that owns the weight, so an update is a local transaction and needs no global schedule (addressing the asynchrony constraint). And because the transpose is ordinary digital arithmetic fed by a forward-directed message, the analog array remains unidirectional (addressing C1).

The last point deserves emphasis, because it is where the "forwards-only" claim could be misconstrued as bookkeeping. The claim is physical and specific to the analog path: no current is ever driven backwards through a transconductor, no array is bidirectional, and nothing waits on a counter-propagating analog signal or on a switch from a free phase to a clamped one. The digital

interconnect carries typed packets in both directions, as any network does; what *forwards-only* forbids is a reverse *analog* channel, not a reverse *packet*. The error is not a backward channel but a forward message that happens to be about the past, and decoupling it from the forward pass in this way is what allows the signalling to be decoupled from a clock.

### 3.4 Transpose-at-source

Mathematically, transposition is a re-indexing: summing a weight matrix down its columns rather than across its rows. Physically, it determines where the weights must be. There is more than one way to compute it in an analog system. A resistive or transconductance crossbar can in principle be driven in reverse to evaluate $\boldsymbol{W}^\top \boldsymbol{\delta}$ in the same array that computes the forward pass — the analog transpose used by some in-memory and SNN designs. We do not take this route, because it makes the array bidirectional (C1) and couples the two directions' operating points; instead we compute the transpose *digitally, on the chip that owns the weights*. Both are legitimate options, and the digital choice is what keeps the analog cell strictly one-way.

That the transpose is digital is not a concession but the design's premise. The analog array exists for *inference*, where the forward MAC sets the power and area budget; learning is a training-time addition. Computing $\boldsymbol{W}^\top \boldsymbol{\delta}$ digitally — only during training, at 6-bit precision, on one engine time-multiplexed across the four cells — adds a small, amortised cost that never touches the inference path, and in exchange keeps the analog array strictly one-way. The question this paper answers is therefore not *analog versus digital learning* but whether an analog inference substrate can be made to learn in place without a reverse analog channel, a host, or a clock. Energy and area are not quantified here: this is a pre-silicon study of the learning mechanism and its accuracy, and a power comparison awaits the physical design.

An earlier revision of this architecture computed $\boldsymbol{W}^\top \boldsymbol{\delta}$ in the router, which therefore held a shadow copy of every attached chip's weights and re-synchronised that copy after every learning sweep. That design met the forwards-only and asynchrony criteria in letter but not in spirit: it carried continuous weight traffic, a coherence barrier, and a drift failure mode. Placing the transpose on the source chip inverts the dataflow. Rather than weights flowing outward to a computing router, the downstream error flows inward on the existing fabric and each chip emits a finished partial. The router retains no weights. The chip's four cells make this efficient: one transpose engine is time-multiplexed across the cells, amortising a single datapath over the whole die, and the engine reads the live SRAM, so the transpose uses the current weights by construction rather than by protocol.

One assumption is introduced. The forward pass and the subsequent transpose may read weights that differ, because learning is continuous. We treat this as a claim to be tested rather than argued, and measure it in Sec. 6. Structurally it is bounded: weights change only at a periodic sweep, and a chip's sweep cannot begin until the current batch's error has been consumed, so the within-chip skew is zero by construction and only cross-chip skew is at issue.

### 3.5 The network

At network level (Fig. 3) the router accumulates the partials addressed to each destination and divides by that destination's nominal fan-in. Forward activations and error messages traverse the same physical links in opposite directions as ordinary packets; no dedicated backward network exists. The two message classes are given deliberately different service guarantees, which Sec. 5 justifies. Because the router alone determines the connectivity, changes of topology — wider layers, deeper stacks, alternative fan-in — are a matter of routing configuration rather than redesign.

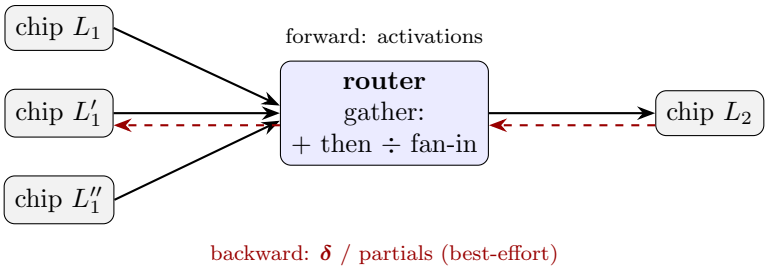


**Figure 3: The network.** The chips perform all computation; the router only gathers. Forward activations (solid) and error messages (dashed) share the same physical links, so there is no separate backward network. Because the transpose runs on the chips, the router holds no weights: it sums the partials arriving for each destination and divides by the nominal fan-in — an adder tree and a divider. Forward traffic is reliable; error traffic is best-effort.

# 4 Formulation

## 4.1 Forward and error recursion

Write $\boldsymbol{x}_0$ for the input feature vector and, for layers $\ell = 1, \ldots, L$,

$$\boldsymbol{a}_\ell = \boldsymbol{W}_\ell \boldsymbol{x}_{\ell-1}, \qquad \boldsymbol{x}_\ell = \varphi(\boldsymbol{a}_\ell), \tag{1}$$

with $\varphi$ a leaky rectifier ($\varphi(u) = u$ for $u \geq 0$, $\alpha u$ otherwise, $\alpha = 0.1$). Each $\boldsymbol{W}_\ell$ is block-structured by the chip factoring of Sec. 3: a chip owns a set of $16 \times 16$ blocks and no block spans chips. A digital read-out $\boldsymbol{s} = \boldsymbol{W}_f \boldsymbol{x}_L + \boldsymbol{b}_f$ produces class scores, and the output error for label $y$ is

$$\boldsymbol{e} = \text{onehot}(y) - \hat{\boldsymbol{p}}, \tag{2}$$

$\hat{\boldsymbol{p}}$ being the read-out's class estimate. One further quantity is read from the same scores: a scalar *happiness* $h$, the read-out's margin quantised to a few levels (0–7), large only when a sample is misclassified or is correct by a thin margin and zero when it is comfortably correct. It carries no per-unit information — one number per sample — and is broadcast to scale the update severity (Sec. 4.3); pinning it to a constant recovers a fixed learning rate. The error is then transported by the standard recursion, evaluated *once* per sample — there is no relaxation phase, which is how C2 is discharged:

$$\boldsymbol{\delta}_L = \mathcal{Q}\Big[\, \mathcal{A}_L\Big( (\boldsymbol{W}_f^\top \boldsymbol{e}) \odot \varphi'(\boldsymbol{a}_L) \Big) \Big], \tag{3}$$

$$\boldsymbol{\delta}_{\ell-1} = \mathcal{Q}\Big[\, \mathcal{A}_{\ell-1}\Big( (\boldsymbol{W}_\ell^\top \boldsymbol{\delta}_\ell) \odot \varphi'(\boldsymbol{a}_{\ell-1}) \Big) \Big]. \tag{4}$$

Equation (4) is exactly backpropagation's recursion; the architectural content is *where* each factor is evaluated. The product $\boldsymbol{W}_\ell^\top \boldsymbol{\delta}_\ell$ is computed on the chip that owns $\boldsymbol{W}_\ell$, from its own SRAM; the Jacobian factor $\varphi'(\boldsymbol{a}_{\ell-1})$ is a sign test on a locally latched activation; and the sum over the chips contributing to a destination is the router's gather. No unit needs a weight it does not own.

The two operators are the hardware. $\mathcal{Q}$ is a fixed-range $b$-bit quantiser modelling the error DAC's rails ($b = 6$ here); $\mathcal{A}_\ell$ is a slow per-layer automatic gain control that tracks the running RMS of the layer's error with an exponential moving average and rescales to a fixed target. The AGC is not cosmetic. The transported error attenuates by $\approx 0.72\times$ per layer — the classical vanishing-gradient factor [17], set by the weight norm — so that under a *fixed-range* quantiser the error eventually underflows the LSB and $\mathcal{Q}$ maps it identically to zero, at which point the layer

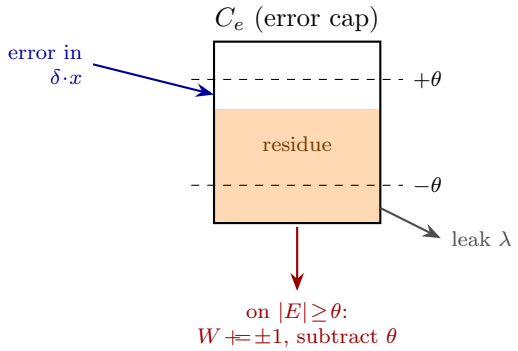


**Figure 4: The leaky jug.** Error accumulates on the capacitor; it leaks, which supplies momentum; a threshold crossing fires one weight LSB and subtracts $\theta$, retaining the residue. The retained residue is what makes the mechanism a sigma–delta modulator rather than a lossy quantiser.

stops learning. Measured usable depth is 4 layers without AGC and beyond 12 with it; the AGC is what makes the quantised error path deep-scalable, and a 6-bit path with AGC outperforms a 10-bit path without.

### 4.2 The update, and why a naive one fails

The per-sample gradient contribution at layer $\ell$ is the outer product $\boldsymbol{g}_\ell = \eta\, \boldsymbol{\delta}_\ell \boldsymbol{x}_{\ell-1}^\top$, accumulated in the error store $\boldsymbol{E}_\ell$. The question is how $\boldsymbol{E}$ is transferred to a weight cell of resolution $\Delta = 1/64$.

Let $g_t$ denote the increment for one synapse at step $t$. The naive transfer, $W \mathrel{+}= \mathrm{round}(g_t/\Delta)$, fails absolutely rather than gracefully: with $|g_t| \approx 0.019\Delta$ the rounding returns zero for every $t$, and the synapse never moves. Nor is this rescued by a larger $\eta$; Sec. 6 reports the rule to be insensitive across a $5\times$ range of $\eta$. The failure is one of *resolution*, and the standard fix buys resolution with per-synapse digital memory (the master weight of C3).

### 4.3 The leaky jug as a sigma–delta modulator

The alternative is to buy resolution with *time*. The error store is a capacitor $C_e$ that is never reset. It leaks; incoming error tops it up; and when it crosses a threshold $\theta$ the synapse *fires*: the weight code moves by one LSB and $\theta$ is *subtracted* from the capacitor, retaining the residue (Fig. 4). For one synapse, per sweep $t$,

$$E_t = \lambda E_{t-1} + g_t, \qquad s_t = \mathrm{sign}(E_t)\,\mathbf{1}[|E_t| \geq \theta]\,, \qquad W_t = W_{t-1} + s_t, \qquad E_t \leftarrow E_t - s_t\theta. \tag{5}$$

The structure is a first-order sigma–delta (delta–sigma) modulator, standard in data conversion and previously used to code activations in neural networks [18]. What is specific to this work is its *role* — it is the weight-update path, which is what lets a coarse analog cell learn with no per-synapse accumulator — and the training-specific corollaries the bounded-error property yields for the hardware below; the modulator itself is not new.

**Bounded, non-accumulating quantisation error.** Take $\lambda = 1$ and sum (5) over $T$ sweeps. Whatever the comparator decides, the update to $E$ and the update to $W$ are the *same* event, so the identity

$$\theta \sum_{t=1}^{T} s_t \;=\; \sum_{t=1}^{T} g_t \;-\; E_T \tag{6}$$

holds exactly. The left side is $\theta$ times the total motion of the weight code; the first term on the right is the total accumulated gradient. Since the subtraction removes exactly $\theta$ whenever the level reaches $\theta$, and the injection per sweep is small, $|E_T|$ remains bounded by the capacitor's clamp $E_{\max}$. Therefore

$$\Big|\sum_{t=1}^{T} s_t - \frac{1}{\theta}\sum_{t=1}^{T} g_t\Big| \;\le\; \frac{E_{\max}}{\theta}, \tag{7}$$

where $\sum_t s_t$ is the total motion of the weight code in LSB — a bound *independent of* $T$. The weight tracks the accumulated gradient scaled by $1/\theta$, with an error that does not grow with the length of training. This is the sigma–delta property, and it is the mathematical content of the design: rounding each update discards a fixed fraction of every step (here, all of it); the jug defers steps but discards none.

Three corollaries follow directly, and each is confirmed in Sec. 6.

*$\theta$ is the learning rate.* By (7) the weight moves $\sum_t g_t/\theta$ codes, so $\theta$ sets the effective step and is the parameter to tune. It is a comparator reference, written once at calibration — the same category of quantity as the ADC gain — and is never discovered at runtime.

*The comparator may be wrong.* Identity (6) does not assume $s_t$ is correct. A wrong-signed fire moves the weight the wrong way *and* pushes the residue further from zero by $\theta$, which makes the next fire more likely and of the correct sign; the charge is conserved and the error is repaid. What must be accurate is the *charge on the capacitor*, not any decision made about it. This inverts the usual analog design burden: the comparator, normally a precision component, becomes the loose one.

*Device mismatch acts as a learning-rate spread.* If $\theta$ or $\lambda$ varies from synapse to synapse, (7) still holds with that synapse's own $\theta_i$. A leaky or high-threshold synapse does not compute a wrong direction; it fires *less often*, i.e. it has a smaller per-synapse learning rate. Mismatch therefore lands on the one axis stochastic gradient descent is known to tolerate — a random diagonal rescaling of the step — rather than on the gradient's sign. Matching is consequently a non-requirement, which is unusual for an analog array and is the property most relevant to yield. This is the same end that in-the-loop surrogate-gradient training achieves for spiking substrates [6], reached here without a backward pass and with the deep credit assignment host-free (Sec. 8).

**One tight specification, and only one.** The mechanism asks for analog accuracy in exactly one place: the residue subtraction must remove a fixed charge $\theta$ independent of the capacitor's voltage, which is realised as a current-steered pulse $Q = It$. Identity (6) is the reason it is the *only* tight specification — because charge is conserved across the fire event, everything downstream of it, the comparator decision included, is permitted to be loose.

**The leak is momentum.** For $\lambda < 1$, (5) unrolls to $E_t = \sum_k \lambda^{t-k} g_k$, which is the exponentially-weighted velocity of a momentum optimiser. The capacitor's leak and the optimiser's velocity register are the same object, so the register is not implemented but inherited. The physical leak available is far slower than needed — $\approx 50\,\mu$V/s on 100 fF, i.e. $\approx 140$ s per error LSB, against a mean inter-fire interval of tens of milliseconds — so on the firing timescale the capacitor is a pure integrator, which is the best-performing configuration measured. The requirement is thus a loose one-sided bound ($\tau_{\text{leak}} \gg$ inter-fire interval, satisfied by $10^3$) and not a servo.

**The comparator is shared and swept, not per-synapse.** A single comparator walks the array, so a synapse is tested once per sweep and can fire at most once per sweep. This is a design

commitment rather than an approximation: it is 256× cheaper per cell, and the sweep period acts as a refractory interval. A per-synapse comparator would allow repeated firing on a single crossing; we measure this to be unstable (Sec. 6). A synapse holding $3\theta$ therefore emits a train of three single fires on successive sweeps rather than one triple fire — the boost is delayed, never discarded. Clamping the capacitor harder would be the wrong rate limiter, as it destroys charge that (6) requires.

**The error is out of the forward path.** $C_e$ is compared, never summed into the MAC. An earlier cell placed a second tail transistor driven by $C_e$ so that the array computed $\boldsymbol{W} + \boldsymbol{E}$. That coupling is a positive feedback loop — error inflates activations, which inflate errors — and Sec. 6 shows it degrades accuracy at the old operating point and collapses the network beyond it. Removing it returns the cell to the original single-tail topology and dissolves a biasing constraint, so the simulator and the silicon compute the same function.

# 5 Interconnect and asynchrony

The message classes are given different guarantees, and the asymmetry is earned rather than assumed. Forward activations are delivered *reliably*, because a forward pass needs its complete input vector. Error messages and transpose partials are *best-effort*: they are fired into accumulators, and a dropped partial contributes zero. Equation (6) is what licenses this — a lost or late error underfills a jug that self-corrects over subsequent sweeps, whereas a lost activation corrupts a prediction irrecoverably.

This is where C4 is discharged in practice. The interconnect carries typed messages, and the type — graded packet, event, or spike — is a property of the fabric, not of the learning rule; a spiking link and a graded-packet link are interchangeable as far as (6) is concerned, since all the jug requires is that charge arriving over time be conserved.

What makes the fabric clock-free is a single structural fact: the gather is a sum, and addition commutes, so partials may arrive in any order. The router therefore divides by the destination's *nominal* fan-in when the batch tag closes and forwards the result, rather than waiting for every contributor.

A wait-for-all barrier is a distributed synchronisation primitive and was the last one in the design; fixed-divide removes it, at the cost of slightly under-driving an error when a partial is missing — the tolerated failure mode above. Batch grouping reuses the existing 4-bit sequence field so that partials from different batches cannot blend, and a configurable freshness window (default four batches) bounds buffering.

Within the window the policy is to apply late rather than drop, since a dropped error loses a gradient whereas a late error costs only bounded skew — and Sec. 6 shows that cost to be small. A loose default matters because the fabric is heterogeneous: nodes on different process nodes run at different rates, and a tight window would penalise the slower node on every batch.

# 6 Evaluation

## 6.1 Method

Evidence is organised as three levels, each constraining the one above: Python behaviour (does the rule learn?), Sky130A SPICE (does the analog physics permit it?), and RTL (does the digital

realisation reproduce the arithmetic?). The behavioural rig is reported here; the design has additionally been carried to Sky130A SPICE and to a bit-faithful RTL implementation, whose netlists, listings, and test benches are in the repository, and we draw on those levels only where a specific claim depends on them.

Two Python rigs are used and are kept distinct. A *float* reference implements the topology and the rule in floating point and is not modified for experiments. A *bit-faithful* model quantises the weight cell, the activations, the error path, and the update, and carries the jug. Each rig is reported against its own control, and figures from different rigs are never compared.

One scope note. The linear read-out $(\boldsymbol{W}_f, \boldsymbol{b}_f)$ is fit by a host least-squares step — L-BFGS logistic regression for the final reported number — on the learned features; it is a single-layer classifier on fixed features, not an in-the-loop gradient through the network. Every hidden-layer weight update, which is the subject of this paper, is performed by the on-die rule with no host. Where we say "host-free" below we mean this deep credit assignment; the final read-out is host-fit, as is standard for evaluating learned features.

The task is EMNIST letters [19] (26 classes, full test set). The topology, denoted BIG, is 48 chips: 1152 input features (split-sign encoded) $\rightarrow$ 384 $\rightarrow$ 128 $\rightarrow$ 256, with a digital read-out. Run-to-run spread at the working operating point is 0.7 percentage points (pp), measured over four seeds; we treat differences within that band as indistinguishable and say so where it matters.

### 6.2 Does the forwards-only rule learn?

Table 1 reports the primary result. The forwards-only rule, carrying every hardware constraint simultaneously (6-bit error broadcast, severity-gated read-out error, 4-bit error integrator, 1-bit sign write at the fold, 8-bit weight rail), reaches 82.50%. The comparison that matters is against full backpropagation on the *identical* chip-factored topology with the same 8-bit weight rail, trained with signed SGD and momentum in an independent framework: 82.85%. The gap is 0.35 pp, within the noise band. On this topology the learning-rule question is therefore closed: the constraints cost essentially nothing.

Two further rows are needed to read that number honestly. A linear classifier on the same 1152 input features reaches 77.14%, so the network is contributing 5.4 pp of genuine nonlinear learning rather than inheriting its accuracy from the features. And the same backpropagation reference on a *dense* network of identical widths reaches 89.48%: the 6.6 pp difference is the price of chip factoring — of forbidding cross-chip mixing within a layer — and is the largest single deficit remaining in the design. It is a question of width and connectivity — topology, not of the learning rule, and it is the subject of ongoing work.

**Table 1:** Learning rule, float rig, EMNIST letters, BIG topology.

| Configuration | Accuracy |
|---|---|
| Backpropagation, dense network, same widths | 89.48% |
| Backpropagation, chip-factored + 8-bit rail (signSGD + momentum) | **82.85%** |
| **Forwards-only rule, all hardware constraints** | **82.50%** |
| Forwards-only rule, unconstrained error broadcast | 82.89% |
| Linear classifier on the 1152 input features (baseline) | 77.14% |
| Prior rule (fold/absorb), best of an extended search | 64.09% |

### 6.3 Does the rule survive the hardware arithmetic?

Carried onto a bit-faithful model — the same rule with the weight cell, activations, error path, and update all quantised and the jug in place — accuracy is 81.96% against a floating-point ceiling of 82.13% for that model (with the measured weight-cell curve and pre-distortion in place, 81.46%; Sec. 3.2). The move to hardware arithmetic therefore costs 0.17 pp, a quarter of the noise band, and it does so while *deleting* the digital master weight, the velocity register, and all per-synapse digital state that a conventional sub-LSB accumulator (C3) would require. A naive write of the same rule to the 8-bit cell, by contrast, reaches only 75.25%: the sigma–delta mechanism, not extra learning rate, is what closes the gap — the working configuration fires on only 0.91% of synapse-sweeps — and $\theta$ behaves as the effective learning rate that (7) predicts.

### 6.4 Robustness

Table 2 tests the three corollaries of Sec. 4.3 against deliberately hostile device models. Leakage was modelled as log-normal per synapse (subthreshold leakage is exponential in threshold voltage, whose mismatch is Gaussian), drawn once as fabrication rather than noise. At $\tau = 100$ folds with a 3× spread the median synapse drains about as fast as it fills and the worst 5% leak six times faster than they fire; the cost is 0.43 pp. A 10× spread costs 0.58 pp. Doubling the spread of $\theta$ costs 0.76 pp. Every figure is at or near the noise band, as (7) predicts, because each perturbation is a per-synapse learning rate rather than a corrupted direction. The frozen fraction rises substantially (from 35% to 66% at $\tau = 100$) and accuracy holds regardless: the network does not need every synapse, only those with evidence.

The comparator result is the sharpest test of the theory. Forcing the fire decision to take the wrong sign on 20% of crossings costs 0.34 pp. A component allowed a one-in-five error rate is not a precision component, and this is a direct consequence of identity (6) rather than a fortunate empirical finding.

A counter-test of allowing the error capacitor to contribute to the forward MAC costs 1.5 pp at the former design point and collapses the network entirely (47 → 12%) at twice that coupling, with the firing rate running away to 19.3% per fold; this is why the cell is single-tail. Permitting multiple fires per crossing (a per-synapse comparator) drives the firing rate to 271% per fold and collapses accuracy to 18%; this is why the comparator is swept.

**Table 2:** Robustness of the jug (bit-faithful rig, BIG). Clean reference 81.96%; noise band 0.7 pp.

| Perturbation | Accuracy | Cost |
|---|---|---|
| None (clean devices) | 81.96% | — |
| Leakage $\tau = 1000$ folds, 3× log-normal spread | 81.68% | 0.28 |
| Leakage $\tau = 100$ folds, 3× spread | 81.53% | 0.43 |
| Leakage $\tau = 1000$ folds, 10× spread | 81.38% | 0.58 |
| Threshold $\theta$ mismatch ×2 | 81.20% | 0.76 |
| Comparator sign wrong on 20% of fires | 81.62% | 0.34 |
| Seeds ($n = 4$) | 81.38 / 81.63 / 81.96 / 82.17 (mean 81.79) | |

### 6.5 Robustness in the face of asynchronicity

Sec. 3.4 introduced one load-bearing assumption: that the forward pass and the transpose may read different weights. We tested it directly by forcing the transpose to read weights staled by $k$ update batches (Table 3). A one-batch skew — the realistic worst case, since a chip's sweep cannot precede the consumption of that batch's error — is free at two convergence levels. The penalty for gross violations is bounded and does not grow with $k$: $k = 2, 4, 8$ are mutually indistinguishable within the noise band, and uncorrelated staleness at $k = 8$ costs 0.34 pp. This is consistent with the transported error being a *direction* generator feeding a slow charge accumulator, and with an independent probe finding the cosine similarity to the true gradient statistically unchanged by weight drift (0.980 with, 0.973 without). Temporal skew is not a blocker, and the freshness window may accordingly be set loose.

**Table 3:** Forward/transpose weight skew, in update batches (float-topology rig; two convergence levels). $k = 1$ is the realistic bound.

| Skew $k$ (batches) | 0 | 1 | 2 | 4 | 8 | 8 (uncorrelated) |
|---|---|---|---|---|---|---|
| Accuracy (early) | 67.47 | **67.47** | 66.33 | 65.75 | 66.34 | 67.13 |
| Accuracy (later) | 72.43 | **72.43** | 71.00 | — | — | — |

### 6.6 Does the analog physics permit it? (SPICE)

Two transistor-level results support the design's central claims, and both concern the one place the mechanism is demanding. First, the residue subtraction of (5) is realised as a current-steered charge pulse, $Q = It$, and is verified in Sky130A SPICE to remove a *fixed* charge independent of the capacitor's voltage. This is the design's single tight analog specification, and identity (6) is the reason it is the only one: because charge is conserved across the fire event, everything downstream of it is permitted to be loose. Second, the component the theory allows to be loose — the comparator — is, in silicon, comfortably precise: a transistor-level window comparator settles to within 0.75 mV of each threshold, with a dead zone of $\approx 0.2\,\mu$V, in 2.5 ns. The robustness measurement above shows the sigma–delta absorbs *wrong* comparator decisions at the level of tens of percent; the physical comparator errs at the level of microvolts. The tolerance the theory demands and the margin the physics delivers are therefore separated by orders of magnitude. (A bit-faithful RTL realisation independently reproduces the transpose within $\pm 1$ LSB and every residue of (5) within $2\,\mu$V of the model; netlists and test benches are in the repository.)

## 7 Design history and negative results

The design's history is reported because several of its dead ends are informative and two of them are methodological.

**Bidirectional schemes were abandoned first.** The work began with a contrastive Hebbian arrangement in which each chip exchanged signals in both directions with its parent. It functioned, but pipelining data in two directions forced a central clock and a management layer to keep the directions aligned — an arrangement that becomes harder, not easier, as clock distribution degrades across a growing fabric and represented increasing fragility and parameter sensitivity in the design. This is C1 and C2 of Sec. 2.2 met head-on, and it is what produced the forwards-only and asynchrony constraints as invariants rather than preferences.

**A long plateau under a $\boldsymbol{W}+\boldsymbol{E}$ decomposition.** The first forwards-only learning scheme started with a more modular W+E design that kept weights $\boldsymbol{W}$ and an error store $\boldsymbol{E}$ per chip, updating $\boldsymbol{E}$ per sample and folding it into $\boldsymbol{W}$ periodically. We explored it extensively — two- and three-layer topologies, frozen and staged training, and a range of feedback signals including a deliberately minimal global scalar that still supports learning. It learned, but repeatedly stalled near 64%: freezing one layer inhibited others, and error routed around the $\boldsymbol{W}+\boldsymbol{E}$ loop tended toward wrong directions or oscillation. The reversals had identifiable causes — a non-stationary objective, since the read-out was refit each epoch; fixed-magnitude sign steps, which orbit rather than converge; and training only on misclassified samples, which starves as the baseline improves.

**On method.** The hardware gap was ultimately closed by elimination rather than hypothesis. Every quantiser was ablated individually and each was innocent — weight cell 0.4 pp, ADC 0, error channel 0, error accumulator 0 — which appeared to be failure and was not: it identified the update path, the one component never isolated. For instance, hypotheses chosen because they were plausible (crest factor, ADC width, analog cell precision) were all wrong in practice or required over-complex hardware. The 'leaky-jug' accumulator was discovered because other options failed.

## 8 Related work and positioning

The design sits between three lines of analog-learning research, and is best understood by what it takes from each and what it refuses.

**Analog in-memory training on non-volatile memory.** Training in analog memory arrays has reached digital-equivalent accuracy [4], but it runs conventional backpropagation — including the transposed backward pass we avoid — and leans on device-level symmetry of the weight update. A related line, resistive processing units [20], makes exactly the digital-accumulator-and-periodic-transfer choice we identify as C3's master weight, and carries its per-synapse memory cost. The sigma–delta update relaxes exactly that requirement: because (6) conserves charge across an imperfect comparator, the write need not be symmetric or linear, only charge-conserving. Our low firing rate (0.91% of synapse-sweeps) also bears on endurance-limited memories, since it reduces write traffic by orders of magnitude relative to update-every-step schemes, though we make no density claim: an analog capacitor is larger than an SRAM byte, and this substrate does not compete on weight density. The strength here is the rule itself: it is not tied to this substrate, and can be applied to other analog cells or even a purely digital accumulator.

**Mixed-signal spiking neuromorphic systems.** The SNN tradition shares our substrate premise — compute as analog sums, communicate as digital events [6, 7, 8, 9]; our C4 (Sec. 2) is precisely that the second half of that premise is a communication choice, not a computational one, and here the error travels forward only, with the hidden-layer learning host-free. On-chip local rules that do dispense with the host [10] remain Hebbian or spike-timing based and, like the on-chip plasticity of Loihi [11], do not reach backpropagation-grade credit assignment on deep tasks; the forwards-only rule does, on this topology, at the cost only of chip factoring.

**Predictive coding and feedback alignment.** Within PC, Whittington and Bogacz [12] establish the backpropagation relationship the rule relies on, and synthesisable digital PC networks have been reported [14]; our aim differs in that the arithmetic is analog and in-memory by construction and the digital logic exists only to move and accumulate error. Feedback alignment and its direct

variant [21, 22] solve the weight-transport problem by discarding the transpose in favour of fixed random feedback — a legitimate and arguably cheaper answer to C1, and one compatible with this fabric. We retain the true transpose because transpose-at-source makes it *free of transport*: the weights are already on the die that needs them, so the accuracy risk of a random feedback path buys nothing here. A direct comparison on this substrate is future work. Equilibrium propagation [23], another local rule that approximates backpropagation, is a natural analog candidate but reintroduces the iterative relaxation (C2) this design abandons.

## 9 Limitations and conclusion

**Limitations.** All results are pre-silicon: Python, SPICE and RTL, not a fabricated die. The largest known accuracy deficit is not the learning rule but the chip factoring, which costs 6.6 pp against a dense network of the same widths (Table 1); widening the fabric is the obvious lever and is untested at scale under the current rule. The jug's threshold is evaluated on a batch grid in simulation, whereas the hardware evaluates it per sweep; a per-sample leak model is required before a circuit specification is frozen. We have modelled static device mismatch but not thermal drift during training. The weight capacitor's own leakage is treated by SRAM refresh, and the inter-chip protocol is specified but not yet realised across the heterogeneous process nodes the architecture anticipates. The supervised rule is evaluated on a single task (EMNIST letters) with a linear read-out — Part I established the substrate on MNIST and EMNIST under unsupervised feature learning [1], but the supervised rule here is characterised on one dataset only; its behaviour on harder problems — in particular whether the momentum the leak supplies earns its keep, which it does not measurably do here — is open.

**Conclusion.** We have described an analog predictive-coding substrate that can be trained in place under three constraints usually treated as obstacles: no calibration, no reverse signal path, no global clock. The design reduces to a small, self-contained predictive-coding level in a W&E design — weights held as capacitor charge with SRAM refresh, an error capacitor whose threshold crossings adjust those weights by one code at a time, and a local transpose that relays error to the level below. Its two enabling ideas are independent of this substrate and may be of wider use. Computing the transpose at the source removes the weight copy, and with it the coherence and synchronisation the copy demands. And the sigma–delta update converts training from a problem of *precision* into a problem of *time* — fire occasionally and conserve charge — which is the trade a capacitor makes naturally, and which leaves the comparator free to be wrong. Against the mixed-signal SNN systems this substrate is adjacent to, the distinction is not that it tolerates mismatch — adaptive analog learning already does — but that it reaches backpropagation-grade accuracy with the error travelling forward only, the deep credit assignment host-free (only the final linear read-out is host-fit), and no global clock. Under these mechanisms a forwards-only network of such chips learns as well as backpropagation on the same topology, while every weight stays where it is.

## Acknowledgements

Architectural direction and design decisions are the author's. The simulators, SPICE netlists, and RTL were implemented with Claude Code [24] (Opus and Fable models), whose contribution was the rate at which code and tests could be produced to explore the topology, algorithm, and parameter space and to characterise the physics.

## References


[1] S. Dobney, "Scalable modular analog VLSI for predictive coding networks: Temporal multiplexing, learned routing, and zero weight bandwidth," *arXiv preprint*, 2026, companion paper (Part I); design files at https://github.com/dobneyresearch/PredictiveCodingNetworks_AnalogVLSIdesign.

[2] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014, pp. 10–14.

[3] C. Mead, *Analog VLSI and neural systems*. Addison-Wesley, 1989.

[4] S. Ambrogio, P. Narayanan, H. Tsai *et al.*, "Equivalent-accuracy accelerated neural-network training using analogue memory," *Nature*, vol. 558, no. 7708, pp. 60–67, 2018.

[5] T. D. Sanger, "Optimal unsupervised learning in a single-layer linear feedforward neural network," *Neural Networks*, vol. 2, no. 6, pp. 459–473, 1989.

[6] B. Cramer, S. Billaudelle, S. Kanya, A. Leibfried, A. Grübl, V. Karasenko, C. Pehle, K. Schreiber, Y. Stradmann, J. Weis, J. Schemmel, and F. Zenke, "Surrogate gradients for analog neuromorphic computing," *Proceedings of the National Academy of Sciences*, vol. 119, no. 4, p. e2109194119, 2022.

[7] E. Chicca, F. Stefanini, C. Bartolozzi, and G. Indiveri, "Neuromorphic electronic circuits for building autonomous cognitive systems," *Proceedings of the IEEE*, vol. 102, no. 9, pp. 1367–1388, 2014.

[8] B. V. Benjamin, P. Gao, E. McQuinn, S. Choudhary, A. R. Chandrasekaran, J.-M. Bussat, R. Alvarez-Icaza, J. V. Arthur, P. A. Merolla, and K. Boahen, "Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations," *Proceedings of the IEEE*, vol. 102, no. 5, pp. 699–716, 2014.

[9] S. Moradi, N. Qiao, F. Stefanini, and G. Indiveri, "A scalable multicore architecture with heterogeneous memory structures for dynamic neuromorphic asynchronous processors (DYNAPs)," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 12, no. 1, pp. 106–122, 2018.

[10] A. Rubino, M. Cartiglia, M. Payvand, and G. Indiveri, "Neuromorphic analog circuits for robust on-chip always-on learning in spiking neural networks," in *IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2023.

[11] M. Davies, N. Srinivasa, T.-H. Lin *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[12] J. C. Whittington and R. Bogacz, "An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity," *Neural computation*, vol. 29, no. 5, pp. 1229–1262, 2017.

[13] B. Millidge, A. Seth, and C. L. Buckley, "Predictive coding: a theoretical and experimental review," *arXiv preprint arXiv:2107.12979*, 2022.

[14] T. Oh, "A synthesizable RTL implementation of predictive coding networks," *arXiv preprint arXiv:2603.18066*, 2026.

[15] R. P. Rao and D. H. Ballard, "Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects," *Nature neuroscience*, vol. 2, no. 1, pp. 79–87, 1999.

[16] R. Bogacz, "A tutorial on the free-energy framework for modelling perception and learning," *Journal of mathematical psychology*, vol. 76, pp. 198–211, 2017.

[17] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.

[18] P. O'Connor and M. Welling, "Sigma delta quantized networks," *arXiv preprint arXiv:1611.02024*, 2016.

[19] G. Cohen, S. Afshar, J. Tapson, and A. Van Schaik, "EMNIST: Extending MNIST to handwritten letters," in *International Joint Conference on Neural Networks (IJCNN)*, 2017, pp. 2921–2926.

[20] T. Gokmen and Y. Vlasov, "Acceleration of deep neural network training with resistive cross-point devices: Design considerations," *Frontiers in Neuroscience*, vol. 10, p. 333, 2016.

[21] T. P. Lillicrap, D. Cownden, D. B. Tweed, and C. J. Akerman, "Random synaptic feedback weights support error backpropagation for deep learning," *Nature Communications*, vol. 7, no. 1, p. 13276, 2016.

[22] A. Nøkland, "Direct feedback alignment provides learning in deep neural networks," in *Advances in Neural Information Processing Systems*, vol. 29, 2016.

[23] B. Scellier and Y. Bengio, "Equilibrium propagation: Bridging the gap between energy-based models and backpropagation," *Frontiers in Computational Neuroscience*, vol. 11, p. 24, 2017.

[24] Anthropic, "Claude Code," https://claude.ai/code, 2025, aI-assisted software engineering tool.